

TECHBIZ SECURITY

Cyber Security Internship Program

Week 2 – Task 5 (EH)

Networking Tools & Subnetting Basics

Submitted By	MANOJ
Role	Cyber Security Intern
Internship Organisation	TechBiz Security
Internship ID	TBS-INT-2026-518
Submitted Date	29 August 2026

Table of Contents

1. Introduction
2. Objectives
3. Tools and Environment
4. Practical Task 1 – Ping
5. Practical Task 2 – Traceroute
6. Practical Task 3 – Nslookup
7. Practical Task 4 – Dig
8. Practical Task 5 – Netstat
9. Subnetting Basics
10. Network vs Host Bits
11. TryHackMe – DNS in Detail
12. TryHackMe – HTTP in Detail
13. TryHackMe Achievement Evidence
14. Observations and Learning Outcomes
15. Security Relevance
16. Challenges and Observations
17. Conclusion
18. Evidence Summary

1. Introduction

This report documents the work completed during Week 2, Task 5 (EH) of the TechBiz Security Cyber Security Internship, focused on Networking Tools and Subnetting Basics. Before any form of network scanning, vulnerability assessment or penetration testing can be performed, an analyst must be comfortable with the fundamental tools used to test connectivity, resolve DNS names, inspect active network connections, and understand how IP addressing and subnetting work. This task covers the practical use of ping, traceroute, nslookup, dig and netstat on a Kali Linux system, basic CIDR-based subnetting concepts, and the completion of two TryHackMe learning rooms covering DNS and HTTP fundamentals.

A solid grasp of these fundamentals directly supports SOC operations (recognising normal versus abnormal network behaviour), vulnerability assessment and penetration testing (reconnaissance and target enumeration), and general troubleshooting during security investigations. All findings in this report are based strictly on the terminal and TryHackMe screenshots captured during the task.

2. Objectives

- Understand basic network connectivity testing.
- Use ping to test reachability and observe latency.
- Use traceroute to understand the path toward a destination.
- Use nslookup and dig for DNS resolution and information gathering.
- Use netstat to inspect active and listening network connections.
- Understand CIDR notation and basic subnetting.
- Calculate usable hosts in a /24 network.
- Complete the required TryHackMe DNS in Detail and HTTP in Detail learning activities.

3. Tools and Environment

The practical exercises in this task were carried out on a Kali Linux system using standard command-line networking utilities, alongside two guided TryHackMe learning rooms. Table 1 lists the tools and platforms actually demonstrated in the evidence gathered for this task.

Tool / Platform	Purpose
Kali Linux	Operating system used to run all command-line networking utilities.
ping	Tests host reachability and measures round-trip response time.
traceroute	Identifies the network path and intermediate hops to a destination.
nslookup	Performs DNS name resolution and queries specific DNS record types.
dig	Performs detailed DNS queries and displays full DNS response data.
netstat	Displays active and listening network connections on the local host.
TryHackMe	Guided learning platform used to complete the DNS in Detail and HTTP in Detail rooms.

4. Practical Task 1 – Ping

Ping is a basic network diagnostic utility that sends ICMP Echo Request packets to a target host and listens for ICMP Echo Reply packets. It is used to verify that a host is reachable over the network and to measure round-trip response time. The following command was executed:

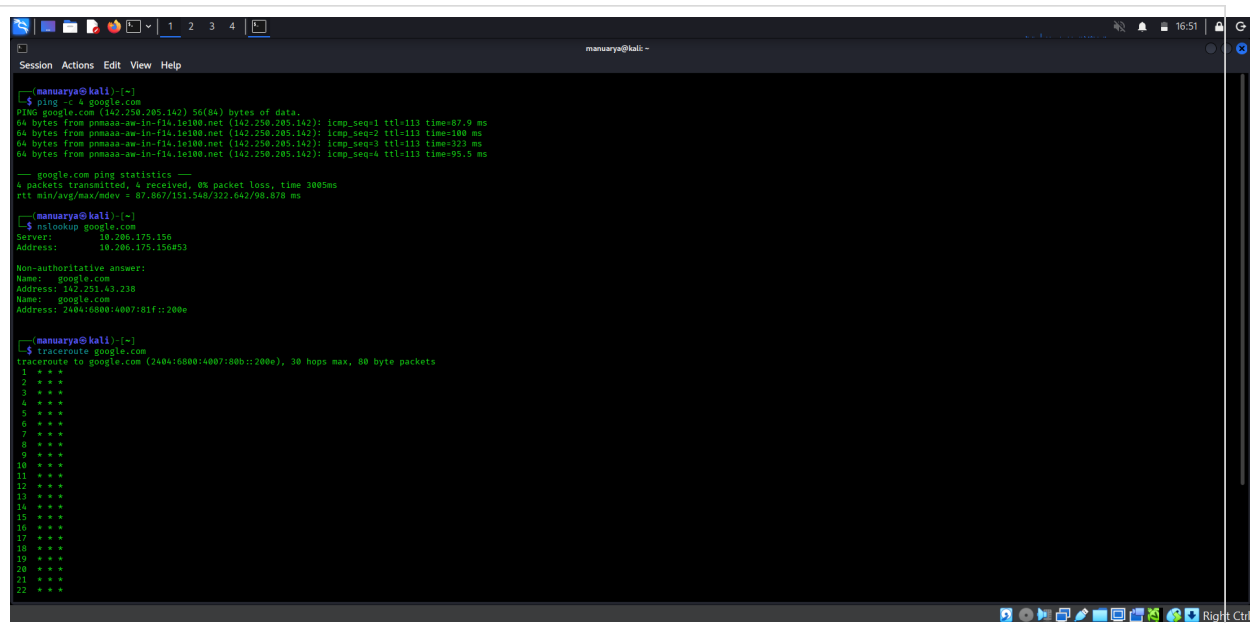
```
$ ping -c 4 google.com
```

Observed Output

The command resolved google.com to the IP address 142.250.205.142 and sent four ICMP echo requests. All four replies were received, with individual response times of 87.9 ms, 100 ms, 323 ms and 95.5 ms. The summary statistics confirmed 4 packets transmitted, 4 packets received, 0% packet loss, with a round-trip time (rtt) minimum/average/maximum/mdev of 87.867/151.548/322.642/98.878 ms. Each reply also displayed a TTL (Time to Live) value of 113.

Interpretation

Packet loss indicates the percentage of sent packets for which no reply was received; 0% packet loss confirms that the destination was fully reachable during the test. TTL indicates the number of network hops a packet can traverse before being discarded, and a decreasing TTL value from the original can be used to estimate the number of hops to the destination. Response time (measured in milliseconds) reflects the round-trip latency between the local host and the destination, which can vary due to network congestion or routing path changes, as reflected in the noticeably higher response time (323 ms) observed for the third request.



```
manuara@kali:~$ ping -c 4 google.com
PING google.com (142.250.205.142) 56(84) bytes of data:
64 bytes from pmaaa-aw-in-f14.1e300.net (142.250.205.142): icmp_seq=1 ttl=113 time=87.9 ms
64 bytes from pmaaa-aw-in-f14.1e300.net (142.250.205.142): icmp_seq=2 ttl=113 time=100 ms
64 bytes from pmaaa-aw-in-f14.1e300.net (142.250.205.142): icmp_seq=3 ttl=113 time=323 ms
64 bytes from pmaaa-aw-in-f14.1e300.net (142.250.205.142): icmp_seq=4 ttl=113 time=95.5 ms

--- google.com ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3005ms
rtt min/avg/max/mdev = 87.867/151.548/322.642/98.878 ms

manuara@kali:~$ nslookup google.com
Server:      10.200.175.156
Address:     10.200.175.156#53

Non-authoritative answer:
Name:   google.com
Address: 142.251.43.238
Name:   google.com
Address: 2404:6800:4007:81f::200e

manuara@kali:~$ traceroute google.com
traceroute to google.com (2404:6800:4007:80b::200e), 30 hops max, 60 byte packets
 0  * * *
 1  * * *
 2  * * *
 3  * * *
 4  * * *
 5  * * *
 6  * * *
 7  * * *
 8  * * *
 9  * * *
10  * * *
11  * * *
12  * * *
13  * * *
14  * * *
15  * * *
16  * * *
17  * * *
18  * * *
19  * * *
20  * * *
21  * * *
22  * * *
```

Figure 1 – Ping, DNS Resolution and Traceroute Output

5. Practical Task 2 – Traceroute

Traceroute is used to map the network path a packet takes to reach a destination by sending probe packets with incrementally increasing TTL values and recording the intermediate routers (hops) that

respond. This allows an analyst to identify the route taken across a network and to pinpoint where connectivity issues occur. The following command was executed:

```
$ traceroute google.com
```

Observed Output

The output showed “traceroute to google.com (2404:6800:4007:80b::200e), 30 hops max, 80 byte packets”, followed by a numbered list of hops. The screenshot displays hops 1 through 22, all shown as “* * *”, meaning no response was received or displayed for those probes. This can occur when intermediate routers are configured to drop or not respond to traceroute probes (commonly due to ICMP/UDP filtering or rate limiting), or due to firewall restrictions along the path. Because the captured output does not show a completed route to the destination, this report does not claim a specific number of successful hops or a confirmed end-to-end path.

Reference: the traceroute output is part of Figure 1 above, in the same terminal session as the ping and nslookup commands.

6. Practical Task 3 – Nslookup

DNS (Domain Name System) resolution is the process of translating a human-readable domain name into an IP address that computers use to route traffic. nslookup is a command-line tool used to query DNS servers and retrieve this information. The following command was executed:

```
$ nslookup google.com
```

Observed Output

The output showed the configured DNS server used for the query, Server: 10.206.175.156, Address: 10.206.175.156#53. Under the non-authoritative answer, two records were returned for google.com: an IPv4 address (142.251.43.238) and an IPv6 address (2404:6800:4007:81f::200e). This demonstrates that DNS can return both IPv4 (A) and IPv6 (AAAA) records for the same hostname. No additional DNS records beyond those visible in the screenshot are claimed in this report.

Reference: the nslookup output is part of Figure 1 above.

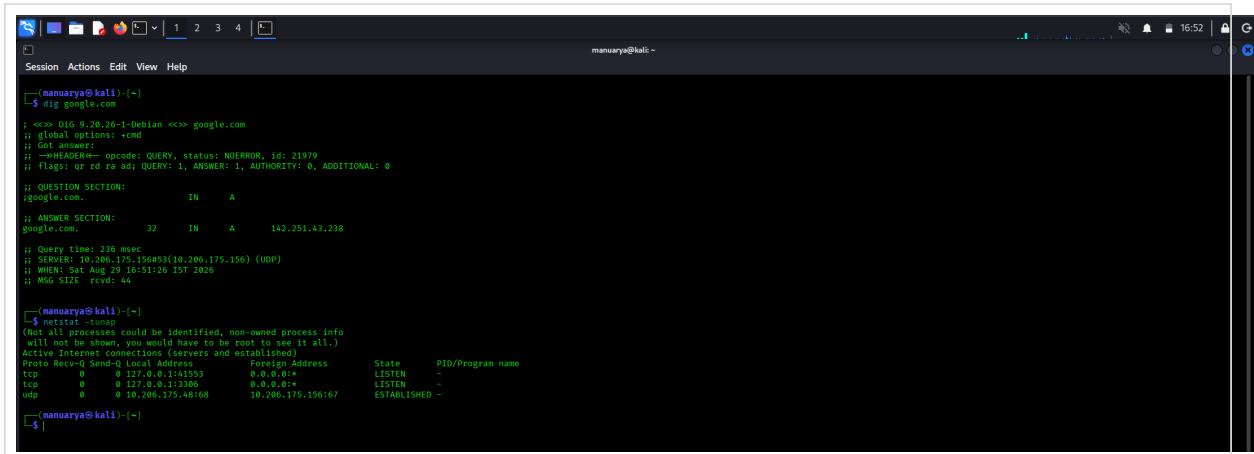
7. Practical Task 4 – Dig

dig (Domain Information Groper) is a flexible DNS query and troubleshooting tool that provides more detailed output than nslookup, including the full structure of the DNS query and response. The following command was executed:

```
$ dig google.com
```

Observed Output

The output was generated using DiG 9.20.26-1-Debian and showed a status of NOERROR, confirming the query was successful. The QUESTION SECTION showed the query “google.com. IN A”, requesting the IPv4 address (A record) for the domain. The ANSWER SECTION returned an A record: “google.com. 32 IN A 142.251.43.238”, matching the IPv4 address observed in the earlier nslookup query. The output also displayed a query time of 236 msec, the DNS server used (10.206.175.156#53, over UDP), the timestamp of the query (Sat Aug 29 16:51:26 IST 2026), and a message size of 44 bytes received.



```

manuarya@kali:~$ dig google.com
; <<< dig 9.20.26-1-Debian <<< google.com
; global options: +cmd
; Got answer:
; --HEADER-- opcode: QUERY, status: NOERROR, id: 21979
; flags: qr rd ra ad; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 0
; QUESTION SECTION:
; google.com.                IN      A
; ANSWER SECTION:
; google.com.                32      IN      A      142.251.43.238
;
; Query time: 236 msec
; SERVER: 10.206.175.156#53(10.206.175.156) (UDP)
; WHEN: Sat Aug 29 16:51:26 IST 2026
; MSG SIZE  rcv= 44

manuarya@kali:~$ netstat -tunap
(Not all processes could be identified, non-owned process info
will not be shown, you would have to be root to see it all.)
Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name
tcp        0      0 127.0.0.1:41553         0.0.0.0:*               LISTEN      -
tcp        0      0 127.0.0.1:3306         0.0.0.0:*               LISTEN      -
udp        0      0 10.206.175.48:68       10.206.175.156:67      ESTABLISHED -

```

Figure 2 – dig DNS Query and netstat Output

8. Practical Task 5 – Netstat

netstat is a command-line utility used to display active network connections, listening sockets, the protocol in use, local and foreign (remote) addresses, and the connection state. This information is useful for identifying which services are exposed on a host and what external connections are currently active. The following command was executed:

```
$ netstat -tunap
```

Observed Output

Before the connection table, the output displayed a warning: “(Not all processes could be identified, non-owned process info will not be shown, you would have to be root to see it all.)”, indicating that the command was not run with elevated privileges and that some process ownership details could not be resolved. Under “Active Internet connections (servers and established)”, two TCP entries were shown in the LISTEN state — 127.0.0.1:41553 and 127.0.0.1:3306 — both bound to the local loopback address with no associated process name displayed. A single UDP connection was also shown in the ESTABLISHED state, with local address 10.206.175.48:68 and foreign address 10.206.175.156:67, consistent with DHCP client/server traffic. No process names or additional connections beyond those visible in the screenshot are claimed in this report.

Reference: the netstat output is part of Figure 2 above, shown in the same terminal session as the dig query.

9. Subnetting Basics

An IPv4 address is a 32-bit value, conventionally written as four decimal octets (for example, 192.168.1.0). CIDR (Classless Inter-Domain Routing) notation expresses how many of those 32 bits are reserved for identifying the network, with the remainder used to address individual hosts within that network. A /24 notation means that the first 24 bits are network bits, leaving 8 bits for host addressing.

Worked Example: 192.168.1.0/24

Calculation Step	Value
Host bits	$32 - 24 = 8$
Total addresses	$2^8 = 256$
Network address	192.168.1.0
Broadcast address	192.168.1.255
Usable host range	192.168.1.1 to 192.168.1.254
Usable hosts	$256 - 2 = 254$

Of the 256 total addresses in a /24 network, two addresses are reserved and cannot be assigned to hosts: the network address (192.168.1.0), which identifies the subnet itself, and the broadcast address (192.168.1.255), which is used to send traffic to every host on the subnet simultaneously. This leaves 254 usable addresses that can be assigned to individual hosts.

10. Network vs Host Bits

Within a 32-bit IPv4 address, the network bits identify which subnet an address belongs to, while the host bits identify a specific device within that subnet. A /24 subnet mask reserves the first 24 bits for the network portion, meaning every device on that subnet shares the same first three octets (for example, 192.168.1.x). The remaining 8 bits are used to uniquely address each host within the subnet, allowing for 256 total combinations, of which 254 are usable host addresses after accounting for the network and broadcast addresses. Understanding this division is fundamental to designing and interpreting network ranges during reconnaissance and scanning.

11. TryHackMe – DNS in Detail

The TryHackMe room “DNS in Detail” was completed as part of this task, covering how DNS works and the different DNS record types used to resolve and support internet services.

Observed Evidence

Figure 3 shows the Task 5 Practical section of the room, with the banner confirming “Room completed: 100%”. The practical used a browser-based DNS query tool alongside a terminal showing the equivalent nslookup commands. The visible questions and correct answers were: “What is the CNAME of shop.website.thm?” answered shops.myshopify.com; “What is the value of the TXT record of website.thm?” answered THM{7012BBA60997F35A9516C2E16D2944FF}; and “What is the numerical priority value for the MX record?” answered 30. The accompanying terminal output confirmed these

results directly: `nslookup --type=CNAME shop.website.thm` returned a canonical name of `shops.myshopify.com`, `nslookup --type=TXT website.thm` returned the TXT record text shown above, and `nslookup --type=MX website.thm` returned a mail exchanger record of “30 `alt4.aspmx.l.google.com`”. Figure 4 shows the room overview page, confirming “DNS in Detail” with “Room completed: 100%”, providing additional completion evidence for the room.

Relevance

This practical activity demonstrated hands-on use of different DNS record types — CNAME (canonical name/alias records), TXT (text records, often used for verification or metadata) and MX (mail exchanger records, used to direct email delivery) — reinforcing the DNS resolution concepts applied earlier with `nslookup` and `dig`.

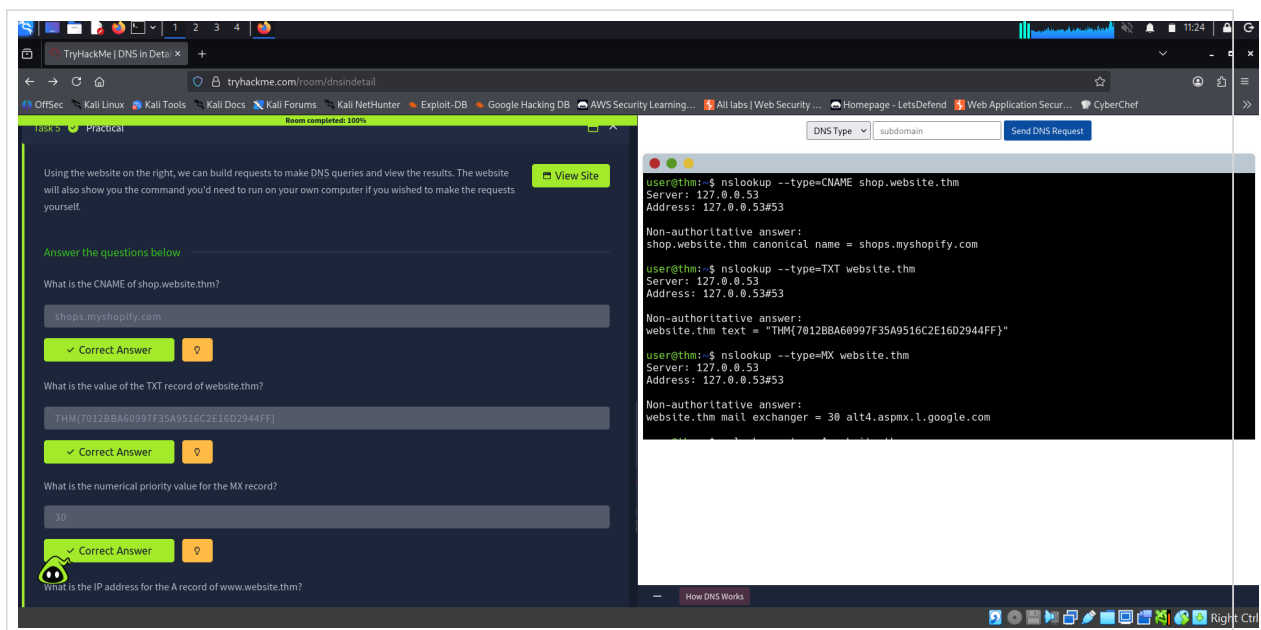


Figure 3 – TryHackMe DNS in Detail Practical Completion Evidence

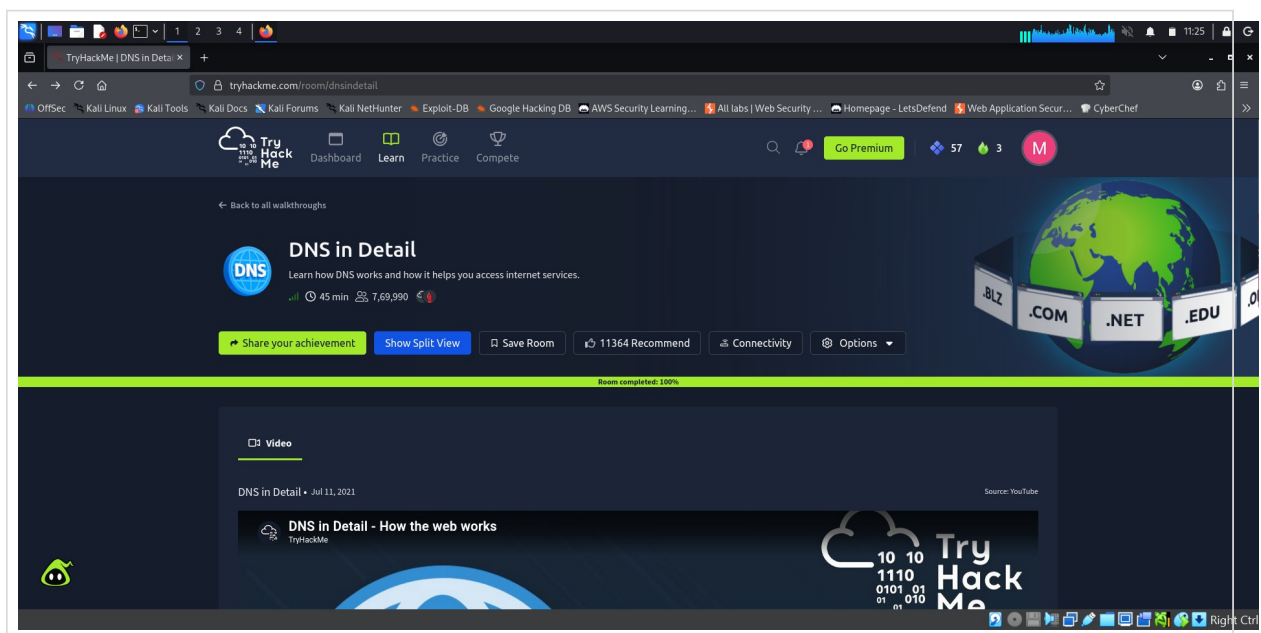


Figure 4 – TryHackMe DNS in Detail Room Completion Evidence

12. TryHackMe – HTTP in Detail

The TryHackMe room “HTTP in Detail” was also completed, covering how content is requested from a web server using the HTTP protocol.

Observed Evidence

Figure 5 shows the room with the banner confirming “Room completed: 100%”, alongside a set of correctly answered HTTP status-code questions. The visible questions and correct answers were: “What response code might you receive if you’ve created a new user or blog post article?” answered 201; “What response code might you receive if you’ve tried to access a page that doesn’t exist?” answered 404; “What response code might you receive if the web server cannot access its database and the application crashes?” answered 503; and “What response code might you receive if you try to edit your profile without logging in first?” answered 401.

Relevance

These questions demonstrate practical understanding of key HTTP status-code categories: 201 Created (successful resource creation), 404 Not Found (requested resource does not exist), 503 Service Unavailable (server temporarily unable to handle the request, as also illustrated by the simulated “Service Unavailable” error page shown in the same screenshot), and 401 Unauthorized (authentication required to access the resource). Understanding these status codes is directly relevant to web application troubleshooting and to interpreting server responses during security testing.

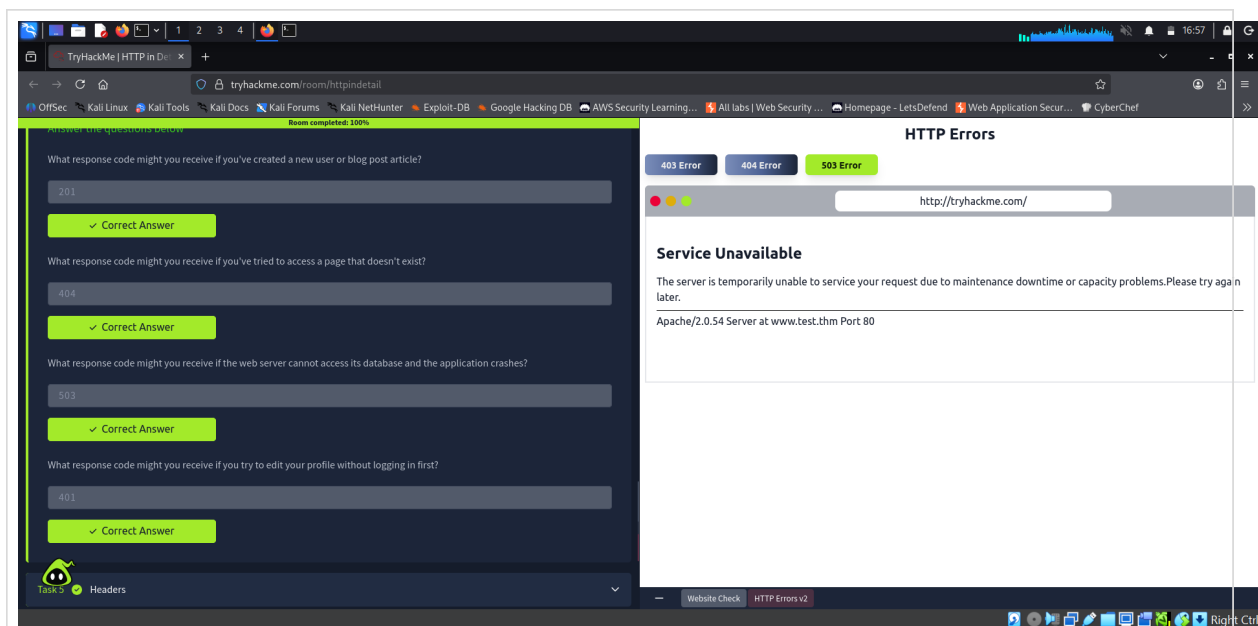


Figure 5 – TryHackMe HTTP in Detail Completed Room

13. TryHackMe Achievement Evidence

Figure 6 provides additional completion evidence for the HTTP in Detail room, showing the room overview page with the banner “Room completed: 100%”, confirming that the room was fully completed in addition to the individual question responses documented in Section 12.

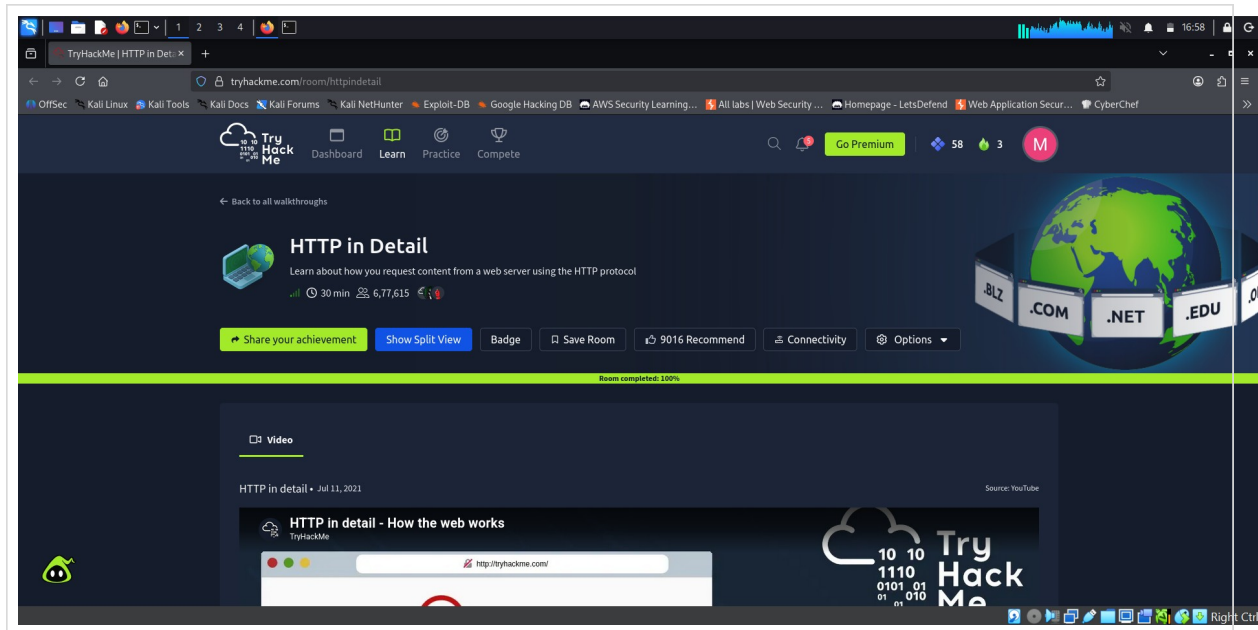


Figure 6 – TryHackMe HTTP in Detail Achievement/Completion Evidence

14. Observations and Learning Outcomes

- Network reachability can be tested using ping, which reports packet loss and round-trip response time.
- DNS names can be resolved using nslookup and dig, both of which returned consistent A record results for google.com.
- Traceroute helps understand the path toward a destination, though intermediate hops may not always respond to probes.
- netstat provides visibility into network connections and listening services on the local host.
- CIDR notation determines the division between network and host bits within an IP address.
- A /24 network contains 256 total IPv4 addresses and 254 usable host addresses after reserving the network and broadcast addresses.
- DNS and HTTP fundamentals are important for cybersecurity reconnaissance, troubleshooting, SOC analysis, and web security.

15. Security Relevance

The networking fundamentals covered in this task form a foundation for a range of defensive and offensive cybersecurity activities:

- Network reconnaissance: identifying live hosts and mapping network paths before further assessment.
- Service identification: using tools such as netstat to understand what services are running and listening on a host.
- DNS investigation: using nslookup and dig to enumerate DNS records, which is a common early step in both reconnaissance and incident investigation.
- Connectivity troubleshooting: using ping and traceroute to diagnose reachability and routing issues.

- SOC alert investigation: correlating DNS queries, connection states and HTTP response codes when reviewing security alerts.
- Vulnerability assessment: understanding exposed services and network ranges before scanning for weaknesses.
- Penetration testing: applying subnetting knowledge to correctly scope target IP ranges during authorized testing.

16. Challenges and Observations

During the traceroute test, all displayed hops (1 through 22) returned “* * *”, indicating that responses were not received or displayed from those intermediate hops. This is a common outcome when routers along the path are configured to drop or rate-limit ICMP/UDP traceroute probes, and does not necessarily indicate a network fault. Separately, the netstat command produced a warning that not all process information could be identified because the command was not executed with root privileges, which limited visibility into which processes owned the listed connections. No other issues were observed in the captured evidence.

17. Conclusion

This task provided practical exposure to fundamental networking commands, DNS analysis, network connection inspection, and CIDR/subnetting concepts, supplemented by TryHackMe-based learning on DNS and HTTP fundamentals. Working through ping, traceroute, nslookup, dig and netstat on a Kali Linux system demonstrated how each tool contributes a different piece of network visibility, from basic reachability testing to detailed DNS record inspection and local connection monitoring. The subnetting exercise reinforced how CIDR notation determines the division between network and host addressing, using the standard /24 example of 256 total addresses and 254 usable hosts.

These skills form a foundational building block for future network scanning, SOC analysis, vulnerability assessment and penetration testing (VAPT) activities to be undertaken later in the internship.

18. Evidence Summary

Evidence	Activity	Result
01_Ping_NSlookup_Traceroute_Google.png	ping, nslookup, traceroute	Successfully demonstrated
02_DNS_Dig_and_Netstat.png	dig and netstat	Successfully demonstrated
03_DNS_in_Detail_Practical.png	TryHackMe DNS in Detail – Practical	Completed
04_DNS_in_Detail_Completed.png	TryHackMe DNS in Detail – Room completion	100% completed
05_thm_http_in_detail_completed.png	TryHackMe HTTP in Detail	100% completed
06_thm_http_in_detail_achievement.png	HTTP in Detail completion evidence	Completion verified